

CV32E40P Front-End Script Study Guide

작성 목적: CV32E40P 프로젝트의 Synopsys front-end script를 공부용으로 읽을 수 있게 해설한다.

이 문서는 단순 명령어 나열이 아니다. 각 script가 flow에서 어떤 역할을 하는지, 주요 줄이 왜 필요한지, constraint와 report가 무엇을 검증하는지까지 연결해서 본다.

0. 전체 flow 한 장 요약

```
RTL / wrapper / filelist
↓
[DC] analyze → elaborate → link smoke test
↓
[DC Topographical] SDC 적용 → compile_ultra -spg -gate_clock
↓
pre-DFT mapped netlist / DDC / SDC / SDF / SVF 생성
↓
[Formality R2N] RTL(reference) ↔ synthesized netlist(implementation) equivalence
↓
[DFT Compiler] scan protocol 정의 → DFT DRC → insert_dft
↓
post-DFT netlist / DDC / SDC / SDF / SPF / SVF 생성
↓
[Formality N2N] pre-DFT netlist(reference) ↔ post-DFT netlist(implementation) equivalence
↓
[TetraMAX] post-DFT netlist + SPF 기반 scan DRC / stuck-at ATPG
↓
[PrimeTime] pre/post-DFT SDF STA 재검증
```

프로젝트의 핵심은 이거다.

DC에서 만든 결과를 그대로 믿지 않고,
FM / DFT / ATPG / PT가 각각 다시 검증하게 만든다.

1. 디렉터리와 stage 의미

configs/	공통 library setup
filelists/	DC/Formality가 읽을 RTL 목록
constraints/	SDC timing constraint
2_Synthesis/	DC synthesis
2.5_FM_R2N/	Formality RTL-to-netlist
3_DFT/	DFT Compiler scan insertion
4_ATPG/	TetraMAX ATPG
5_FM_N2N/	Formality netlist-to-netlist
6_STA/	PrimeTime STA
00_Project_Tracking/	실행 결과/결정/상태 요약

2.5_FM_R2N 이 synthesis와 DFT 사이에 있는 이유:

DFT 넣기 전에 pre-DFT netlist가 RTL과 같은지 먼저 확인해야 한다.
이게 통과해야 이후 DFT/N2N 결과도 의미가 있다.

2. 공통 library setup

파일:

```
configs/library_setup.tcl
```

원본:

```
set SAED32_ROOT /DATA/home/edu135/lib/SAED32_EDK

set RVT_TT_DB $SAED32_ROOT/lib/stdcell_rvt/db_nldm/saed32rvt_tt1p05v25c.db
set LVT_TT_DB $SAED32_ROOT/lib/stdcell_lvt/db_nldm/saed32lvt_tt1p05v25c.db
set HVT_TT_DB $SAED32_ROOT/lib/stdcell_hvt/db_nldm/saed32hvt_tt1p05v25c.db

set_app_var target_library [list $RVT_TT_DB $LVT_TT_DB $HVT_TT_DB]
set_app_var link_library [list * $RVT_TT_DB $LVT_TT_DB $HVT_TT_DB]
```

줄별 의미

줄	의미	왜 필요한가
<code>set SAED32_R00T</code> <code>...</code>	SAED32 library root path 고정	모든 <code>.db</code> , Milkyway, TLU+ path의 기준점
<code>RVT_TT_DB</code>	regular-Vt standard cell timing library	기본 cell mapping 후보
<code>LVT_TT_DB</code>	low-Vt timing library	빠른 cell 후보. timing closure에 유리하지만 leakage 증가 가능
<code>HVT_TT_DB</code>	high-Vt timing library	느리지만 leakage 절감 가능
<code>target_library</code>	DC가 mapping할 수 있는 cell library	RTL operator를 실제 standard cell로 바꾸는 대상
<code>link_library</code>	design/reference resolve용 library	이미 mapping된 cell, submodule, <code>*</code> 현재 design reference를 link하기 위함

핵심 개념

`target_library` 와 `link_library` 는 다르다.

```
target_library = 어떤 cell로 합성할 것인가
link_library   = design 안 reference를 어디서 찾을 것인가
```

`link_library [list * ...]` 의 `*` 의미:

현재 메모리에 읽힌 design들을 먼저 link 대상으로 포함한다.
없으면 RTL submodule끼리 reference resolve가 깨질 수 있다.

주의:

이 setup은 TT 1.05V 25C 단일 corner다.
multi-corner signoff라고 말하면 안 된다.

3. RTL filelist

파일:

```
filelists/cv32e40p_dc.tcl
```

핵심 구조:

```
set RTL_INC_DIRS [list \  
    rtl/cv32e40p/rtl/include \  
]  
  
set RTL_FILES [list \  
    rtl/cv32e40p/rtl/include/cv32e40p_apu_core_pkg.sv \  
    ...  
    rtl/tech/cv32e40p_clock_gate.sv \  
    ...  
    rtl/cv32e40p/rtl/cv32e40p_top.sv \  
    rtl/wrappers/cv32e40p_synth_wrap.sv \  
]
```

왜 filelist가 중요한가

SystemVerilog는 compile order가 중요하다.

package → primitive/tech wrapper → lower modules → top → synth wrapper

순서가 틀리면 이런 문제가 난다.

```
package type unresolved  
module reference unresolved  
parameter/type not found  
clock gate wrapper missing
```

중요한 줄

항목	의미
RTL_INC_DIRS	include , package import 경로
cv32e40p_*_pkg.sv	typedef, enum, parameter가 들어간 package. 먼저 읽어야 함
rtl/tech/cv32e40p_clock_gate.sv	synthesis용 clock-gating wrapper. DFT/FM에서 중요
cv32e40p_core.sv , cv32e40p_top.sv	실제 CPU hierarchy
cv32e40p_synth_wrap.sv	synthesis/DFT/STA용 wrapper top

왜 `cv32e40p_top` 이 아니라 **wrapper**를 **top**으로 쓰나

보통 공개 IP top은 SoC integration용 port가 그대로 노출된다. 합성/DFT/STA에서는 test port, scan clock-gating enable, tie-off, constraint-friendly boundary가 필요하다.

그래서 wrapper를 둔다.

```
cv32e40p_top = 원본 IP top  
cv32e40p_synth_wrap = implementation flow용 top boundary
```

4. SDC constraint 해설

파일:

```
constraints/cv32e40p_func_10ns.sdc
```

원본:

```
create_clock -name clk_i -period 10.0 [get_ports clk_i]  
set_clock_uncertainty 0.1 [get_clocks clk_i]  
  
set_case_analysis 0 [get_ports scan_cg_en_i]  
set_case_analysis 0 [get_ports scan_en]  
set_case_analysis 0 [get_ports scan_in]  
  
set_false_path -from [get_ports rst_ni]  
  
set_input_delay 1.0 -clock clk_i [get_ports {...}]  
set_input_delay 0.0 -clock clk_i [get_ports scan_in]  
set_input_delay 0.0 -clock clk_i [get_ports rst_ni]  
  
set_output_delay 1.0 -clock clk_i [get_ports {...}]
```

줄별 의미

명령	의미	왜 필요한가
<code>create_clock -period 10.0</code>	<code>clk_i</code> 를 10ns clock으로 정의	STA의 기준. 없으면 reg-to-reg path를 계산 못 함
<code>set_clock_uncertainty 0.1</code>	clock jitter/skew margin 0.1ns 반영	ideal clock만 보면 과하게 낙관적이므로 margin 부여
<code>set_case_analysis 0 scan_cg_en_i</code>	functional mode에서 scan clock-gating enable을 0으로 고정	scan/test path가 functional STA를 오염하지 않게 함
<code>set_case_analysis 0 scan_en</code>	functional mode에서 scan shift 비활성화	scan mux가 functional timing과 섞이지 않게 함
<code>set_case_analysis 0 scan_in</code>	scan input을 functional mode constant로 처리	scan input floating/unknown 방지
<code>set_false_path -from rst_ni</code>	async reset 입력에서 출발하는 timing path 제외	reset assertion/deassertion은 별도 recovery/removal 또는 reset strategy로 다룸
<code>set_input_delay 1.0</code>	외부 launching logic이 1ns를 사용한다고 가정	input-to-reg path budget 설정
<code>set_output_delay 1.0</code>	외부 capturing logic에 1ns margin을 준다고 가정	reg-to-output path budget 설정

이 SDC의 성격

이건 production SoC constraint가 아니다.

functional 10ns baseline constraint

즉 목적은:

CPU core 단독 implementation flow를 재현 가능하게 만들기 위한 기준 constraint

주의:

input/output delay 1ns는 SoC 실제 timing contract가 아니라 baseline assumption이다.

5. Analyze / Elaborate / Link smoke test

파일:

```
2_Synthesis/0_Script/run_analyze_elab_link.tcl
```

이 script의 목적:

합성 최적화 전에 RTL을 tool이 제대로 읽고 hierarchy를 만들 수 있는지 확인한다.

코드 흐름

```
set PROJECT_ROOT /DATA/home/edu135/CV32E40P
cd $PROJECT_ROOT
```

프로젝트 root를 고정한다. 상대경로 script가 깨지지 않게 한다.

```
source configs/library_setup.tcl
```

`.db` library를 읽을 준비를 한다. `link` 때 standard cell / design reference가 필요하다.

```
file mkdir ...
define_design_lib WORK -path 2_Synthesis/work
```

DC의 compile/elaborate working library를 만든다. SystemVerilog analyze 결과가 WORK에 저장된다.

```
source filelists/cv32e40p_dc.tcl
set_app_var search_path [concat $search_path $RTL_INC_DIRS]
analyze -format sverilog $RTL_FILES
```

filelist를 읽고 include path를 붙인 뒤 SystemVerilog RTL을 analyze한다.

```
elaborate cv32e40p_synth_wrap
current_design cv32e40p_synth_wrap
link
```

`elaborate` 는 RTL을 generic design hierarchy로 만든다. `current_design` 은 이후 명령 대상 top을 지정한다. `link` 는 내부 module과 library reference를 연결한다.

```
check_design > ...  
report_design > ...
```

합성 전에 기본 design problem을 확인한다.

```
write -format ddc -hierarchy -output ...elab.ddc
```

elaboration 결과를 DDC로 저장한다. 나중에 debug/reload 가능하다.

왜 이 단계가 따로 필요한가

바로 `compile_ultra`로 가면 실패 원인이 모호해진다.

```
RTL parse 문제인지  
filelist 문제인지  
library link 문제인지  
constraint 문제인지  
optimization 문제인지
```

구분이 안 된다. 그래서 smoke test를 먼저 둔다.

6. Normal DC synthesis

파일:

```
2_Synthesis/0_Script/run_compile_10ns.tcl
```

이 script는 non-topographical compile baseline이다.

핵심 차이:

```
run_compile_10ns.tcl      = wire-load 기반 일반 DC compile  
run_compile_10ns_topo.tcl = Milkyway/TLU+ 기반 topographical compile
```

핵심 줄

```
read_sdc constraints/cv32e40p_func_10ns.sdc
```

constraint를 적용한다. 이 줄 전까지는 tool이 clock/input/output timing 목표를 모른다.


```
set_svf 2_Synthesis/2_Output/svf/cv32e40p_synth_wrap.pre_dft.svf
compile_ultra -gate_clock
set_svf -off
```

`set_svf` 는 Formality용 guide file을 켜다. `compile_ultra` 가 수행한 register move, clock gating, hierarchy optimization 같은 변환 정보를 SVF에 기록한다.

`-gate_clock` 은 clock gating optimization을 허용한다.

주의:

SVF 없이는 Formality가 DC 최적화 의도를 모를 수 있다.
특히 clock gating이나 sequential-looking transform이 있으면 match/verify가 어려워진다.

```
report_qor
report_timing
report_constraint -all_violators
report_area -hierarchy
report_power -hierarchy
```

합성 결과를 숫자로 남긴다.

report	보는 것
<code>report_qor</code>	WNS/TNS/area 등 요약
<code>report_timing</code>	critical path 상세
<code>report_constraint -all_violators</code>	max cap/transition/timing 등 violation
<code>report_area -hierarchy</code>	hierarchy별 area
<code>report_power -hierarchy</code>	hierarchy별 power estimate

```
write_ddc / write_verilog / write_sdc
```

후속 tool용 artifact를 만든다.

artifact	후속 사용처
<code>.ddc</code>	DC/DFT Compiler reload
<code>.vg</code>	Formality/PrimeTime/TetraMAX input
<code>.sdc</code>	PT/ICC2 등 constraint handoff
<code>.svf</code>	Formality guide

7. Topographical DC synthesis

파일:

```
2_Synthesis/0_Script/run_compile_10ns_topo.tcl
```

현재 포트폴리오에서 더 중요한 script다.

왜 **topographical compile**을 쓰나

일반 DC는 wire delay를 wire-load model로 추정한다. 작은 design에서는 가능하지만, 실제 backend와 차이가 커질 수 있다.

Topographical mode는 Milkyway physical library와 TLU+ RC tech를 이용해 placement-aware delay를 추정한다.

일반 compile = 논리 중심 추정

Topo compile = 물리 정보를 일부 반영한 합성

Physical collateral 설정

```
set TECH_FILE ...saed32nm_1p9m_mw.tf
set TLUPLUS_MAX ...Cmax.tluplus
set TLUPLUS_MIN ...Cmin.tluplus
set TLUPLUS_MAP ...tf_itf_tluplus.map
```

변수	의미
TECH_FILE	layer/via/design rule 등 Milkyway technology file
TLUPLUS_MAX	worst RC extraction table
TLUPLUS_MIN	best RC extraction table
TLUPLUS_MAP	tech file layer와 ITF/TLU+ layer mapping

```
set MW_RVT ...
set MW_LVT ...
set MW_HVT ...
set MW_DESIGN_LIB 2_Synthesis/mw_lib/cv32e40p_topo_mw
```

Milkyway reference library와 design library를 지정한다.

```
file delete -force $MW_DESIGN_LIB
create_mw_lib \
  -technology $TECH_FILE \
  -mw_reference_library [list $MW_RVT $MW_LVT $MW_HVT] \
  -hier_separator {/} \
  -bus_naming_style {%d} \
  -open $MW_DESIGN_LIB
```

기존 MW design lib를 지우고 새로 만든다. 반복 실행 시 stale library 때문에 이전 결과가 섞이는 걸 막는다.

```
set_tlu_plus_files ...
check_tlu_plus_files
check_library
```

RC tech와 library consistency를 검사한다. Topo compile 전에 꼭 필요하다.

Compile 부분

```
read_sdc constraints/cv32e40p_func_10ns.sdc
set_svf ...pre_dft_topo.svf
compile_ultra -spg -gate_clock
set_svf -off
```

-spg 는 Synopsys Physical Guidance 관련 option이다. 물리 정보를 활용한 compile 품질을 높이는 목적이 다.

`-gate_clock` 은 clock gating 삽입/최적화를 허용한다. CV32E40P wrapper의 scan clock-gating enable 과 FM constant 처리에 영향을 준다.

Output

```
write -format ddc ...pre_dft_topo.ddc
write -format verilog ...pre_dft_topo.vg
write_sdc ...pre_dft_topo.sdc
write_sdf ...pre_dft_topo.sdf
```

Topo 결과는 일반 compile보다 후속 flow에 더 많이 쓰인다.

output	의미
<code>pre_dft_topo.ddc</code>	DFT Compiler input
<code>pre_dft_topo.vg</code>	Formality/PrimeTime input
<code>pre_dft_topo.sdc</code>	post-synthesis constraint handoff
<code>pre_dft_topo.sdf</code>	estimated delay annotation
<code>pre_dft_topo.svf</code>	R2N Formality guide

8. Formality R2N

파일:

```
2.5_FM_R2N/0_Script/run_fm_r2n_topo.tcl
```

R2N 의미:

```
Reference = RTL
Implementation = synthesized netlist
```

목적:

합성 후 netlist가 functional mode에서 RTL과 같은지 확인한다.

핵심 줄

```
set_svf $SVF_FILE
```

DC가 남긴 optimization guide를 Formality에 읽힌다. 이게 중요하다.

```
set_verification_clock_gate_reverse_gating true
```

clock gating이 들어간 design에서 reverse gating 처리를 허용한다. clock-gated netlist와 RTL의 equivalence를 맞추기 위해 필요하다.

```
read_db -technology_library [list $RVT_TT_DB $LVT_TT_DB $HVT_TT_DB]
```

implementation netlist 안 standard cell을 해석하기 위한 technology library를 읽는다.

```
source filelists/cv32e40p_dc.tcl
read_sverilog -r -l2 -libname WORK $RTL_FILES
set_top r:/WORK/$TOP_NAME
```

RTL을 reference side로 읽는다. `r:`가 reference container다.

```
set_clock r:/WORK/$TOP_NAME/clk_i
set_constant -type port r:/WORK/$TOP_NAME/scan_cg_en_i 0
set_constant -type port r:/WORK/$TOP_NAME/scan_en 0
set_constant -type port r:/WORK/$TOP_NAME/scan_in 0
```

functional equivalence 조건을 고정한다.

왜 scan 관련 port를 0으로 고정하나:

```
검증하려는 건 normal functional mode다.
scan mode까지 equivalence에 넣으면 DFT/test mux behavior 때문에 비교 의도가 흐려진다.
```

```
set_dont_verify_points -directly_undriven_output
```

직접 구동되지 않는 output point를 verification 대상에서 제외한다. 공개 IP wrapper나 scan/test boundary에서 발생할 수 있는 noise를 줄인다.

```
read_verilog -i -netlist -libname WORK $NETLIST
set_top i:/WORK/$TOP_NAME
```

synthesis netlist를 implementation side로 읽는다. `i:`가 implementation container다.

```
match
verify
```

`match` 는 compare point를 대응시킨다. `verify` 는 실제 equivalence를 증명한다.

report 의미

```
unmatched_points = 대응 실패한 point
passing_points   = equivalence pass point
failing_points   = equivalence fail point
```

포트폴리오에서는 `failing 0` 이 가장 중요하다. 단, unmatched가 있으면 왜 괜찮은지 설명해야 한다.

9. DFT insertion

파일:

```
3_DFT/0_Script/run_insert_dft_10ns_topo.tcl
```

목적:

```
pre-DFT DDC를 읽어서 scan chain을 삽입하고 post-DFT netlist/SPF/SVF를 만든다.
```

입력

```
set PRE_DFT_DDC 2_Synthesis/2_Output/pre_dft_topo/cv32e40p_synth_wrap.pre_dft_topo.ddc
```

DFT Compiler는 DC에서 만든 DDC를 읽는다. Verilog netlist보다 DDC가 tool 내부 정보가 풍부해서 DFT 작업에 유리하다.

Topographical DFT 환경

DFT script도 Milkyway/TLU+를 다시 설정한다.

```
create_mw_lib ...
set_tlu_plus_files ...
check_tlu_plus_files
check_library
```

이유:

insert_dft 이후 timing/QoR를 물리 추정 기반으로 봐야 하기 때문.

기본 timing constraint

```
create_clock -name clk_i -period 10.0 [get_ports clk_i]
set_clock_uncertainty 0.1 [get_clocks clk_i]
set_false_path -from [get_ports rst_ni]
```

DFT insertion 중에도 clock/reset timing context가 필요하다. 특히 scan FF mux 삽입 후 timing report를 보기 위해 필요하다.

scan configuration

```
set_scan_configuration \
  -style multiplexed_flip_flop \
  -chain_count 1 \
  -clock_mixing no_mix \
  -add_lockup true
```

option	의미
multiplexed_flip_flop	functional DFF 앞에 scan mux를 붙이는 일반 scan style
chain_count 1	scan chain 1개 구성
clock_mixing no_mix	서로 다른 clock domain을 한 chain에 섞지 않음
add_lockup true	clock skew/edge 차이에 대비해 lockup latch 삽입 허용

DFT configuration

```
set_dft_configuration \
  -scan enable \
  -connect_clock_gating enable
```

scan insertion을 켜고, clock gating cell의 test enable 연결도 처리하겠다는 의미다.

DFT signal 정의

```
set_dft_signal -view existing_dft -type ScanClock -port clk_i -timing {45 55}
set_dft_signal -view existing_dft -type Reset -port rst_ni -active_state 0
set_dft_signal -view existing_dft -type TestMode -port scan_cg_en_i -active_state 1
```

`existing_dft` 는 design에 이미 존재하는 DFT 관련 port/clock/reset을 tool에 알려주는 view다.

```
set_dft_signal -view spec -type ScanEnable -port scan_en -active_state 1
set_dft_signal -view spec -type ScanDataIn -port scan_in
set_dft_signal -view spec -type ScanDataOut -port scan_out
set_scan_path chain0 -view spec -scan_data_in scan_in -scan_data_out scan_out
```

`spec` 는 새 scan 구조를 어떻게 만들지 지정하는 view다.

즉:

```
existing_dft = 현재 design에 이런 test control이 있다
spec        = 내가 원하는 scan path는 이렇다
```

DFT DRC와 insert

```
create_test_protocol

dft_drc > pre_dft.drc.rpt
...
set_svf ...post_dft_topo.svf
insert_dft
set_svf -off
```

`create_test_protocol` 은 위에서 정의한 scan clock/reset/test signal을 기반으로 test protocol을 만든다.

`dft_drc` 는 insert 전 scan 가능성 문제를 확인한다.

`insert_dft` 는 실제 scan mux, scan chain, test logic을 삽입한다.

`set_svf` 를 켜 이유:

```
post-DFT netlist가 pre-DFT netlist와 functional mode에서 같은지 N2N Formality로 확인하려면 DFT insertion
변환 정보가 필요하다.
```


post-DFT output

```
write_test_protocol ...spf
write ddc ...post_dft_topo.ddc
write verilog ...post_dft_topo.vg
write_sdc ...post_dft_topo.sdc
write_sdf ...post_dft_topo.sdf
```

output	후속 사용처
<code>.spf</code>	TetraMAX scan DRC/ATPG
<code>.vg</code>	FM N2N, PT STA, ATPG
<code>.ddc</code>	DC/PT/DFT reload
<code>.sdc</code>	STA constraint handoff
<code>.sdf</code>	delay annotation
<code>.svf</code>	FM N2N guide

10. Formality N2N

파일:

```
5_FM_N2N/0_Script/run_fm_n2n_topo.tcl
```

N2N 의미:

```
Reference = pre-DFT synthesis netlist
Implementation = post-DFT scan netlist
```

목적:

scan insertion 이후에도 functional mode 동작이 유지되는지 확인한다.

왜 RTL이 아니라 pre-DFT netlist와 비교하나

DFT insertion은 synthesis netlist에 scan mux/test logic을 추가한다. 따라서 비교 기준은 보통 바로 직전 clean netlist다.

```
RTL → pre-DFT netlist = R2N
pre-DFT netlist → post-DFT netlist = N2N
```

둘 다 pass하면 다음 chain이 성립한다.

```
RTL == pre-DFT netlist == post-DFT netlist(functional mode)
```

핵심 줄

```
set_svf $SVF_FILE
```

DFT insertion에서 생성한 SVF를 읽는다.

```
read_verilog -r -netlist -libname WORK $REF_NETLIST
...
read_verilog -i -netlist -libname WORK $IMPL_NETLIST
```

reference와 implementation이 둘 다 netlist다.

```
set_constant scan_cg_en_i 0
set_constant scan_en 0
set_constant scan_in 0
```

functional mode comparison이다. scan mode behavior를 비교하는 게 아니다.

```
match
verify
```

compare point matching 후 equivalence verify.

면접에서 말할 포인트

```
DFT 삽입은 구조를 크게 바꾸므로, post-DFT netlist를 STA/ATPG로 넘기기 전에 N2N Formality로 functional equivalence를 확인했다.
```

11. TetraMAX ATPG

파일:

```
4_ATPG/0_Script/run_tmax_stuck_at_topo.tcl
```

목적:

post-DFT scan netlist와 SPF를 읽어서 stuck-at fault pattern을 생성하고 coverage를 확인한다.

입력

```
set NETLIST_FILE ...post_dft_topo.vg
set SPF_FILE      ...post_dft_topo.spf
```

file	의미
post-DFT .vg	scan cell이 들어간 gate netlist
.spf	scan chain/test protocol 정보

library/model read

```
read_netlist -library ...saed32nm.tv
read_netlist -library ...saed32nm_lvt.tv
read_netlist -library ...saed32nm_hvt.tv
read_netlist $NETLIST_FILE
```

TetraMAX는 Verilog cell simulation model이 필요하다. .db가 아니라 ATPG용 Verilog model을 읽는다.

build model

```
set_rules B12 ignore
set_learning -atpg_equivalence
run_build_model $DESIGN_NAME
```

run_build_model은 ATPG 내부 model을 구성한다. B12 floating input rule은 첫 pass에서 block하지 않도록 ignore 처리했다.

주의:

ignore/warning은 signoff-clean이 아니라 first-pass ATPG 진행을 위한 분류다.

scan DRC

```
set_drc -allow_unstable_set_resets
set_drc -clock -dynamic -nodisturb_clock_grouping
set_rules Z3 warning
set_contention nowire -severity warning
run_drc $SPF_FILE
```

SPF 기반으로 scan chain이 실제로 shift/capture 가능한지 검사한다.

fault model / ATPG

```
set_faults -fault_coverage
set_faults -model stuck
set_faults -report collapsed
add_faults -all
```

stuck-at fault model을 사용한다. collapsed fault 기준 report를 만든다.

```
set_atpg -capture_cycles 4 -abort_limit 32 -num_processes 4
set_atpg -fill adjacent -coverage 98
set_atpg -merge high -decision random -store
run_atpg
```

ATPG option 의미:

option	의미
capture_cycles 4	capture cycle 수
abort_limit 32	fault당 search 포기 한계
coverage 98	목표 coverage
merge high	pattern merge 적극 적용
store	generated pattern 저장

output

```
write_patterns ...serial.stil
report_faults -summary
write_faults ...
analyze_faults -class UD/AU/ND
```

output	의미
STIL	tester/pattern exchange format
faults.summary	coverage/fault class 요약
UD/AU/ND analysis	undetected/ATPG untestable/not detected 원인 분석

12. PrimeTime STA scripts

파일:

```
6_STA/0_Script/run_pt_pre_dft_10ns.tcl
6_STA/0_Script/run_pt_pre_dft_10ns_sdf.tcl
6_STA/0_Script/run_pt_post_dft_10ns_sdf.tcl
```

12.1 pre-DFT wire-load STA

```
set NETLIST 2_Synthesis/2_Output/pre_dft/cv32e40p_synth_wrap.pre_dft.vg
set SDC_FILE constraints/cv32e40p_func_10ns.sdc
set link_path [list * $RVT_TT_DB $LVT_TT_DB $HVT_TT_DB]
read_verilog $NETLIST
current_design $TOP_NAME
link_design
set_wire_load_mode enclosed
set_wire_load_model -name 70000 [current_design]
read_sdc $SDC_FILE
```

이건 일반 DC netlist를 wire-load model로 STA하는 script다.

실제 physical delay가 아니라 wire-load estimate 기반

12.2 pre-DFT topo SDF STA

```
set NETLIST ...pre_dft_topo.vg
set SDF_FILE ...pre_dft_topo.sdf
read_verilog $NETLIST
link_design
read_sdc $SDC_FILE
read_sdf $SDF_FILE
```

DC topographical synthesis가 만든 SDF delay를 PT에 annotate한다.

왜 필요한가:

DC report만 믿지 않고, 같은 netlist/SDF/SDC를 PT에서 다시 읽어 timing을 재검증한다.

12.3 post-DFT topo SDF STA

```
set NETLIST ...post_dft_topo.vg
set SDF_FILE ...post_dft_topo.sdf
read_sdc $SDC_FILE
read_sdf $SDF_FILE
```

scan insertion 이후 netlist의 functional timing을 다시 확인한다.

중요:

DFT 삽입 후 scan mux가 들어가면 area/timing이 바뀐다.
그래서 post-DFT STA는 별도로 필요하다.

공통 report

```
check_timing -verbose
report_global_timing
report_timing -delay_type max
report_timing -delay_type min
report_constraint -all_violators
report_analysis_coverage
report_annotated_delay
```

report	의미
<code>check_timing</code>	unconstrained endpoint, no_clock, generated clock 문제 등 점검
<code>report_global_timing</code>	WNS/TNS 요약
<code>report_timing -delay_type max</code>	setup path
<code>report_timing -delay_type min</code>	hold path
<code>report_constraint -all_violators</code>	timing/electrical violation 전체
<code>report_analysis_coverage</code>	STA가 얼마나 design을 커버했는지
<code>report_annotated_delay</code>	SDF delay가 얼마나 annotate됐는지

13. Flow에서 꼭 이해해야 할 제약 개념

13.1 Functional mode vs scan mode

이 프로젝트의 SDC/FM은 대부분 functional mode 기준이다.

```
scan_en = 0
scan_cg_en_i = 0
scan_in = 0
```

의미:

CPU가 정상 동작하는 모드에서 equivalence/timing을 본다.

DFT/TetraMAX에서는 scan mode를 따로 정의한다.

```
scan_en = 1
scan_cg_en_i = 1
scan_in/scan_out chain active
```

두 모드를 섞으면 해석이 깨진다.

13.2 Reset false path

```
set_false_path -from [get_ports rst_ni]
```

reset input에서 출발하는 data timing을 제외한다.

이건 reset이 중요하지 않다는 뜻이 아니다.

functional data STA와 async reset protocol 검증은 다른 문제다.

13.3 SDF STA의 의미

SDF는 delay annotation이다.

```
netlist = 구조
SDC      = constraint
SDF      = delay number
```

PT에서 셋을 같이 읽어야 실제 path slack을 계산한다.

13.4 SVF의 의미

SVF는 Formality guide다.

DC/DFT가 design을 어떻게 바꿨는지 FM에게 알려주는 기록

FM이 magic으로 모든 최적화를 이해하는 게 아니다. SVF가 있어야 매칭이 쉬워진다.

13.5 DDC vs VG

파일	성격	주 사용처
<code>.ddc</code>	Synopsys 내부 design DB	DC/DFT reload
<code>.vg</code>	Verilog gate netlist	FM/PT/TetraMAX/외부 handoff

14. 공부 순서 추천

처음부터 DFT/ATPG를 보면 헷갈린다. 이 순서로 보라.

1. configs/library_setup.tcl
2. filelists/cv32e40p_dc.tcl
3. constraints/cv32e40p_func_10ns.sdc
4. 2_Synthesis/0_Script/run_analyze_elab_link.tcl
5. 2_Synthesis/0_Script/run_compile_10ns_topo.tcl
6. 2.5_FM_R2N/0_Script/run_fm_r2n_topo.tcl
7. 3_DFT/0_Script/run_insert_dft_10ns_topo.tcl
8. 5_FM_N2N/0_Script/run_fm_n2n_topo.tcl
9. 4_ATPG/0_Script/run_tmax_stuck_at_topo.tcl
10. 6_STA/0_Script/run_pt_pre_dft_10ns_sdf.tcl
11. 6_STA/0_Script/run_pt_post_dft_10ns_sdf.tcl

각 단계에서 질문은 하나씩만 잡는다.

단계	질문
library	어떤 cell/corner로 mapping하나
filelist	RTL compile order가 왜 이 순서인가
SDC	어떤 mode와 clock을 검증하나
DC	RTL이 어떤 netlist/artifact로 변하나
FM R2N	합성 netlist가 RTL과 같은가
DFT	scan chain을 어떻게 정의하고 삽입하나
FM N2N	scan 삽입 후 functional equivalence가 유지되나
ATPG	scan chain으로 stuck-at fault를 얼마나 detect하나
PT	DC 결과를 독립 STA tool로 재검증했나

15. 면접 설명용 flow 문장

짧게:

CV32E40P 공개 RTL을 wrapper 기준으로 정리한 뒤, SAED32 TT mixed-VT library에서 DC Graphical synthesis를 수행했습니다. 합성 결과는 SVF 기반 Formality R2N으로 RTL 등가성을 확인했고, DFT Compiler로 1-chain muxed scan을 삽입한 뒤 post-DFT netlist를 N2N Formality로 다시 검증했습니다. 이후 TetraMAX stuck-at ATPG와 PrimeTime SDF STA로 scan 구조와 timing을 재확인했습니다.

더 기술적으로:

단순히 DC compile 결과만 본 것이 아니라, pre-DFT topo netlist, SDC, SDF, SVF를 후속 tool handoff artifact로 남겼습니다. R2N/N2N Formality는 scan/test port를 functional constant로 고정해 normal mode equivalence를 검증했고, DFT 단계에서는 scan clock/reset/test mode/scan path를 명시적으로 정의해 SPF를 생성했습니다. PrimeTime에서는 SDF annotation과 analysis coverage를 확인해 DC timing report와 독립적으로 setup/hold를 재검증했습니다.

16. 현재 script의 개선 포인트

공부용/포트폴리오용으로는 좋다. 다만 다음 개선을 하면 더 실무형이다.

16.1 PROJECT_ROOT hardcoded 제거

현재:

```
set PROJECT_ROOT /DATA/home/edu135/CV32E40P
```

개선:

```
if {[info exists ::env(PROJECT_ROOT)]} {  
    error "PROJECT_ROOT env is required"  
}  
set PROJECT_ROOT $::env(PROJECT_ROOT)
```

왜:

다른 machine/path에서 재현 가능해야 한다.

16.2 library path 환경변수화

현재 SAED32 path가 hardcoded다. private path가 공개 문서에 노출될 수 있다.

개선:

```
SAED32_ROOT env 사용  
public repo에는 template만 제공
```

16.3 MCMM 확장

현재 TT 단일 corner다.

다음 단계:

```
SS/FF corner 추가  
setup/hold corner 분리  
scenario별 SDC 정리
```

16.4 Backend handoff 준비

backend로 넘어가려면 아래 artifact와 정보를 정리해야 한다.

```
pre-DFT functional netlist  
post-DFT scan netlist optional  
SDC  
SAED32 Milkyway/NDM/LEF equivalent  
TLU+  
floorplan target utilization  
power/ground net  
tap/endcap/filler/tie cell list
```

17. 안전한 claim boundary

가능한 표현:

```
CV32E40P front-end closure baseline  
DC/DC Graphical synthesis  
Formality R2N/N2N pass  
DFT scan insertion  
TetraMAX stuck-at ATPG  
PrimeTime SDF STA
```

금지 표현:

```
RTL-to-GDS 완료  
post-route signoff 완료  
IR/EM 완료  
production DFT signoff  
multi-corner signoff closure
```

이 프로젝트는 현재 front-end closure로 강하다. backend evidence가 생기면 그때 RTL-to-GDS로 올리면 된다.